AI 数字员工系统架构 — 九大核心问题详解

> 基于对项目代码的深度分析，涵盖 agent 系统核心、数据库 schema、DAL/Actions、工作流引擎等 28+ 文件。

一、用户目标意图的拆解

当前实现

系统通过 **意图解析器 + 工作流模板** 两级机制来拆解用户目标：

第一级：意图解析器（`src/lib/agent/intent-parser.ts`）

```
用户输入 → Claude Sonnet 分析 → 结构化意图 → 推荐执行步骤
```

当用户提交一个创作目标（如"写一篇关于养老金改革的深度报道"），意图解析器：

1. 接收参数：`topicTitle`（主题）、`scenario`（场景）、`availableEmployees`（可用员工列表）
2. 调用 Claude Sonnet 4 进行语义分析
3. 输出结构化意图对象：
    - `intentType`: breaking_news / deep_report / social_campaign / series / event_coverage / routine
    - `scale`: single（单篇）/ batch（批量）/ series（系列）
    - `timeConstraint`: urgent / normal / flexible
    - `requiredCapabilities`: 所需能力列表（如 ["深度分析", "数据调研", "长文写作"]）
    - `suggestedSteps`: 推荐的执行步骤数组，每步指定 {key, label, employeeSlug}

第二级：工作流模板（`src/db/schema/workflows.ts`）

- 预定义 `workflowTemplates` 表存储可复用的步骤编排
- 内置场景模板在 `constants.ts` 的 `AUTO_SCENARIO_TEMPLATES` 中定义
- 支持 4 种预设场景：
    - `breaking_news_auto`: 热点→创作→审核→发布（4 步，无需审批）
    - `event_express`: 赛事监控→裁剪→制作→发布（4 步）
    - `daily_briefing`: 每日 6 点自动新闻简报（5 步，需审批，cron: `0 6 * * *`）
    - `weekly_deep_report`: 周度深度报道（8 步全流程，需审批，cron: `0 9 * * 1`）

执行链路的标准 8 步流水线：

顺序	步骤 Key	中文名称	默认负责员工
1	monitor	热点监控	小雷 (xiaolei)
2	plan	选题策划	小策 (xiaoce)
3	material	素材准备	小资 (xiaozi)
4	create	内容创作	小文 (xiaowen)
5	produce	视频制作	小剪 (xiaojian)
6	review	质量审核	小审 (xiaoshen)
7	publish	渠道发布	小发 (xiaofa)
8	analyze	数据分析	小数 (xiaoshu)

设计思路

意图拆解采用 **AI 推荐 + 人工确认** 模式，而非纯自动化。解析器给出推荐步骤后，用户通过 `StartWorkflowDialog` UI 可以：

- 调整步骤顺序
- 增删步骤
- 更换每步负责员工
- 设置审批节点

如果解析失败，系统回退到 `DEFAULT_STEPS` 保底方案（包含全部 8 步），确保不会因为意图理解错误导致流程无法启动。

待完善

- **F4.1.99-100**: 并行步骤和条件分支尚未实现（当前只支持顺序执行）
- 意图解析目前是一次性的，不支持执行过程中的动态调整

二、数字员工技能的学习

当前实现

技能的"学习"在系统中体现为 **技能绑定 + 熟练度进阶** 的机制，而非传统意义的模型微调。

技能绑定类型（`skillBindingTypeEnum`）：

类型	含义	来源	能否解绑
`core`	核心技能	创建时内置匹配	不可解绑
`extended`	扩展技能	人工绑定/学习获得	可解绑
`knowledge`	知识技能	知识库关联	可解绑

熟练度系统（`employeeSkills.proficiency`）：

- 范围：0-100 的整数
- 初始值：核心技能默认 70，扩展技能默认 50
- 进阶机制：通过 `updateSkillProficiency` action 手动调整
- 影响：注入到系统提示词中，指导 AI 的工作风格：
  - ≤30 (初级): "请保守行事，遵循标准步骤，逐步验证"
  - 31-70 (中级): "平衡创新与稳妥，适当探索新方法"
  - ≥71 (高级): "鼓励创新，可尝试非常规方案"

学习来源：

来源	机制	代码位置
内部知识库	通过 `employeeKnowledgeBases` 关联，知识内容注入系统提示词	`assembly.ts:54-72`
记忆积累	执行后的 feedback/pattern/preference 存入 `employee_memories`	`employee-memories.ts`
技能市场	通过 UI 浏览和绑定新技能（`SkillBrowserDialog`）	`skill-browser-dialog.tsx`
执行日志	`execution_logs` 记录每次执行的输入/输出/指标	`execution-logs.ts`

模式学习（`learnedPatterns`）：

员工的 `learnedPatterns` 字段（JSONB）存储已发现的模式：

```
{
  pattern: string;       // 模式描述
  confidence: number;    // 置信度 0-1
  source: string;        // 来源（feedback/execution）
  discoveredAt: string; // 发现时间
}
```

这些模式通过 DAL 层（`evolution.ts` 的 `getLearnedPatterns()`）读取并展示在员工详情页的"进化"标签页中。

待完善

- **真正的自动学习**: 当前熟练度需人工调整，缺少基于执行成功率的自动进阶逻辑
- **F4.1.134-135 RAG**: 知识库的向量化检索（embedding）基础设施未搭建
- **F4.3.x 自我进化**: 除执行日志外，完整的自学习闭环（自动发现模式→调整行为）尚未实现

三、员工创建时的技能匹配

当前实现

系统支持两种员工创建方式，技能分配策略不同：

方式一：预设员工（`isPreset = true`）— 内置匹配

8 个预设员工在 `EMPLOYEE_CORE_SKILLS` 映射中硬编码了核心技能：

```
// src/lib/constants.ts:289-298
export const EMPLOYEE_CORE_SKILLS: Record<string, string[]> = {
  xiaolei: ["web_search", "trend_monitor", "social_listening", "heat_scoring"],
  xiaoce: ["topic_extraction", "angle_design", "audience_analysis",
"task_planning"],
  xiaozi: ["media_search", "knowledge_retrieval", "news_aggregation",
"case_reference"],
  xiaowen: ["content_generate", "headline_generate", "style_rewrite",
"script_generate"],
  xiaojian: ["video_edit_plan", "thumbnail_generate", "layout_design",
"audio_plan"],
  xiaoshen: ["quality_review", "compliance_check", "fact_check",
"sentiment_analysis"],
  xiaofa: ["publish_strategy", "style_rewrite", "translation",
"audience_analysis"],
  xiaoshu: ["data_report", "competitor_analysis", "audience_analysis",
"heat_scoring"],
};
```

通过 `db:seed`（`src/db/seed.ts`）初始化时：

1. 在 `ai_employees` 表插入 8 个预设员工
2. 在 `skills` 表插入 28 个内置技能
3. 在 `employee_skills` 表建立核心绑定关系（`bindingType = 'core'`）
4. 核心技能初始熟练度默认 70

方式二：自定义员工（`isPreset = false`）— 新学习

通过 `EmployeeCreateDialog` UI 创建自定义员工：

1. 用户填写名称、职位、座右铭
2. 创建时不自动分配技能（技能列表为空）
3. 用户后续通过"技能浏览器"(`SkillBrowserDialog`) 手动选择技能绑定
4. 绑定前系统执行 **兼容性检查**（`F4.1.39`）：验证技能的 `compatibleRoles` 与员工角色匹配

技能绑定流程：

```
用户选择技能 → bindSkillToEmployee() action
  ↓
验证：技能存在？员工存在？已绑定？兼容？
  ↓
插入 employee_skills (bindingType='extended', proficiency=50)
  ↓
创建配置版本快照 (employee_versions)
```

设计逻辑

维度	预设员工	自定义员工
核心技能	创建时自动绑定 4 个	无，需手动选择
技能解绑	核心技能不可解绑	所有技能可解绑
扩展技能	可额外绑定	可绑定任意兼容技能
初始熟练度	核心 70 / 扩展 50	统一 50

四、内置技能包含哪些

完整清单：28 个内置技能，6 大类别

定义位置：`src/lib/constants.ts:193-234`

感知类 Perception（4 个）

Slug	名称	描述	版本
`web_search`	全网搜索	搜索互联网获取最新信息和热点话题	3.2
`trend_monitor`	趋势监控	实时监控 30+ 平台热点趋势变化	2.1
`social_listening`	社交聆听	监测社交媒体舆情和用户讨论	1.8
`news_aggregation`	新闻聚合	聚合多源新闻资讯并去重排序	2.0

分析类 Analysis（6 个）

Slug	名称	描述	版本
`sentiment_analysis`	情感分析	分析文本情感倾向（正面/负面/中性）	2.5
`topic_extraction`	主题提取	从文本中提取核心主题和关键词	2.0
`competitor_analysis`	竞品分析	分析竞品内容策略和表现数据	2.0
`audience_analysis`	受众分析	分析目标受众画像、偏好和行为	1.9
`fact_check`	事实核查	多源交叉验证事实准确性	3.5
`heat_scoring`	热度评分	基于多维数据计算话题热度指数	2.1

生成类 Generation（7 个）

Slug	名称	描述	版本
`content_generate`	内容生成	根据大纲和要求生成高质量内容	4.0
`headline_generate`	标题生成	生成多版本吸引力标题	2.3
`summary_generate`	摘要生成	自动生成文章摘要和提要	2.0
`script_generate`	脚本生成	生成视频/音频脚本和分镜	1.8
`style_rewrite`	风格改写	按指定风格改写内容	2.0
`translation`	多语翻译	支持中英双语互译及本地化	1.5
`angle_design`	角度设计	基于热点设计多个差异化内容角度	2.5

制作类 Production（4 个）

Slug	名称	描述	版本
`video_edit_plan`	视频剪辑方案	生成视频剪辑计划和分镜脚本	2.8
`thumbnail_generate`	封面生成	根据内容自动生成封面设计方案	1.6
`layout_design`	排版设计	自动排版和版式设计建议	1.5
`audio_plan`	音频方案	配音配乐方案和语音合成计划	1.3

管理类 Management（4 个）

Slug	名称	描述	版本
`quality_review`	质量审核	对内容进行全面质量审核评分	3.5
`compliance_check`	合规检查	检测政治、法律、伦理敏感内容	4.0
`task_planning`	任务规划	将复杂任务拆解为可执行步骤	2.0
`publish_strategy`	发布策略	制定多渠道发布时间和策略	3.0

知识类 Knowledge（4 个）

Slug	名称	描述	版本
`knowledge_retrieval`	知识检索	从知识库中检索相关知识片段	2.0
`media_search`	媒资检索	从媒资库中检索素材	3.0
`case_reference`	案例参考	检索历史爆款案例作为创作参考	1.5
`data_report`	数据报告	生成数据分析报告，汇总传播数据	2.6

技能与工具的关系

每个技能 slug 直接映射到 `tool-registry.ts` 中的 AI SDK Tool 定义。当员工执行任务时：

- 只有已绑定的技能对应的工具才会被注入到 agent 上下文
- 权限级别进一步过滤可用工具（observer 无工具，advisor 仅只读工具）

五、数字员工之间的数据交换与协同

当前实现

协同通过 **工作流制品传递 + 团队消息系统** 双通道实现：

通道一：工作流制品传递（核心数据交换）

```
Step 1 (小雷-热点监控)
  ↓ 输出: topic_brief 制品
Step 2 (小策-选题策划)
  ↓ 接收上游制品 + 输出: angle_list 制品
Step 3 (小资-素材准备)
  ↓ 接收上游全部制品 + 输出: material_pack 制品
Step 4 (小文-内容创作)
  ↓ 接收上游全部制品...
```

具体实现在 `execution.ts:35-59`：

```
// 构建用户消息时注入上游上下文
let userContent = `任务: ${step.title}\n场景: ${scenario}\n步骤: ${stepLabel}\n指令: ${step.instruction}`;

// 附加上游步骤摘要
if (previousOutputs.length > 0) {
  userContent += "\n\n--- 前序步骤概要 ---\n";
  previousOutputs.forEach(po => {
    userContent += `[${po.stepKey}] ${po.summary}\n`;
  });

  // 附加详细制品内容
  userContent += "\n--- 前序步骤详细产出 ---\n";
  previousOutputs.forEach(po => {
    po.artifacts.forEach(a => {
      userContent += `\n### ${a.title}\n${a.content}\n`;
    });
  });
}
```

制品存储在 `workflow_artifacts` 表中：

- `artifactType`: topic_brief / angle_list / material_pack / article_draft / video_plan / review_report / publish_plan / analytics_report / generic
- `content`: 制品正文（JSONB 或纯文本）
- 每步可产出多个制品，下游可获取全链路上下文

通道二：团队消息系统（状态同步 + 人机交互）

`team_messages` 表承载的消息类型：

类型	用途	示例
`status_update`	状态变更广播	"小文 开始执行内容创作"
`work_output`	产出物通知	"小文 完成了文章初稿"
`decision_request`	审批请求	"需要审批：深度报道初稿"
`alert`	告警/干预	"小审 发现合规问题"

消息支持结构化附件（`attachments`）：

- `topic_card`：话题卡片（标题、热度、来源）
- `draft_preview`：稿件预览
- `chart`：数据图表
- `asset`：媒资素材

消息支持交互按钮（`actions`）：

```
actions: [
  { label: "批准", value: "approve", variant: "primary" },
  { label: "驳回", value: "reject", variant: "destructive" },
  { label: "修改意见", value: "feedback", variant: "default" }
]
```

通道三：团队规则约束

团队配置（`teams.rules`）定义协同规则：

- `approvalRequired`：是否需要人工审批
- `reportFrequency`：汇报频率（实时/每小时/每4小时/每日）
- `sensitiveTopics`：敏感话题列表，相关内容强制审批
- `approvalSteps`：指定哪些步骤需要审批
- `escalationPolicy`：升级策略（质量阈值、敏感度阈值、超时策略）

六、执行结果判断与修正

结果判断机制

1. 质量自评分（自动）

系统提示词的第 7 层要求每个员工在输出末尾必须附上质量自评：

```
【质量自评：85/100】
```

`step-io.ts` 的 `extractQualityScore()` 解析此分数，记录在制品的 `metrics.qualityScore` 中。

2. 执行状态判断（自动）

`parseStepOutput()` 根据输出内容确定状态：

- `success`：正常完成
- `partial`：部分完成
- `needs_approval`：需要人工审批（observer/advisor 权限的员工自动触发）

3. 升级策略（自动）

`escalationPolicy` 定义在团队配置中：

```
escalationPolicy: {
  qualityThreshold: 70,        // 质量分 < 70 自动升级
  sensitivityThreshold: 0.8,   // 敏感度 > 0.8 自动升级
  timeoutMinutes: 30,          // 超时分钟
  timeoutAction: "auto_approve" | "auto_reject" | "escalate"
}
```

当质量分低于阈值时，系统自动触发人工审核。

4. 人工审批（手动）

工作流引擎（`execute-workflow.ts`）中的审批等待：

```
步骤执行完成
  ↓
检查：需要审批？(teamRules.approvalRequired || output.status === 'needs_approval')
  ↓ 是
发送 decision_request 消息 → 等待 Inngest 审批事件
  ↓
人工操作：approve / reject
```

问题修正机制

机制一：驳回重做（F4.1.106-107）

```
人工驳回 + 反馈意见
  ↓
approveWorkflowStep(stepId, "rejected", "标题不够吸引，需要更有冲击力")
  ↓
工作流引擎重新执行该步骤，反馈意见注入到 agent 上下文
  ↓
员工收到："【驳回反馈】标题不够吸引，需要更有冲击力"
  ↓
基于反馈重新生成输出
```

代码路径：`workflow-engine.ts` 的 `approveWorkflowStep()` → 更新步骤状态为 `pending` → 发送 Inngest 事件 → 工作流引擎重新执行该步骤。

机制二：中途干预

工作流引擎在每步执行前检查是否有人工干预消息：

```
// execute-workflow.ts
// 检查是否有人工干预消息（alert 类型）
const interventions = await getTeamMessages(teamId, { type: 'alert', after:
stepStartTime });
if (interventions.length > 0) {
  // 将干预消息注入到当前步骤的执行上下文
}
```

机制三：执行日志追踪

`execution_logs` 表记录每次执行的完整信息：

- 输入上下文、输出内容、token 用量
- 工具调用次数、执行时长
- 使用的模型和温度参数
- 状态和错误信息

机制四：Token 预算控制

工作流实例设置 `tokenBudget`（默认 50000），引擎在每步执行后累加实际用量：

```
累计 tokens > tokenBudget → 终止工作流 → 通知人工
```

防止失控执行导致成本爆炸。

七、员工记忆系统的构建与隔离

记忆存储模型

`employee_memories` 表结构：

字段	类型	说明
`id`	UUID	主键
`employeeId`	UUID	关联 AI 员工 (FK)
`organizationId`	UUID	组织隔离 (FK)
`memoryType`	enum	feedback / pattern / preference
`content`	text	记忆内容
`importance`	real	重要度 0-1
`accessCount`	integer	访问次数
`lastAccessedAt`	timestamp	最后访问时间
`expiresAt`	timestamp	过期时间（可选）
`createdAt`	timestamp	创建时间

三种记忆类型

类型	来源	示例
`feedback`	人工审批的反馈意见	"标题需要更简洁，控制在15字以内"
`pattern`	执行中发现的规律	"体育类稿件使用短句效果更好"
`preference`	用户偏好设定	"输出格式偏好 Markdown，段落不超过200字"

记忆注入机制

在 agent 组装阶段（`assembly.ts:74-90`），系统：

1. 按 `importance DESC` 排序查询该员工的记忆
2. 取 **Top 10** 条高重要度记忆
3. 格式化为文本注入系统提示词的第 6 层

```
--- 过往经验与记忆 ---
[feedback/重要度:0.9] 标题需要更简洁，控制在15字以内
[pattern/重要度:0.8] 体育类稿件使用短句效果更好
[preference/重要度:0.7] 输出格式偏好 Markdown
...
```

隔离策略

三级隔离：

```
组织级隔离 (organizationId)
  └── 员工级隔离 (employeeId)
        └── 类型级分类 (memoryType)
```

1. **组织隔离**: 所有记忆必须关联 `organizationId`，DAL 查询自动按组织过滤
2. **员工隔离**: 每条记忆绑定特定 `employeeId`，员工 A 无法读取员工 B 的记忆
3. **时间隔离**: `expiresAt` 支持记忆过期自动失效
4. **LRU 淘汰**: `accessCount` 和 `lastAccessedAt` 为未来的记忆淘汰策略提供数据基础

配置版本与记忆的关系

`employee_versions` 表为员工的每次配置变更创建快照（包括记忆重置），支持：

- 完整配置回滚
- 审计追踪（谁在什么时间做了什么修改）

待完善

- **记忆自动写入**: 当前记忆主要靠手动或审批反馈，缺少执行后自动总结学到的教训
- **记忆容量管理**: 没有自动清理过期/低重要度记忆的定时任务
- **跨员工经验共享**: 同一团队的员工无法共享经验（完全隔离）

八、员工与技能的关系

关系模型

```
AIEmployee ←(M:N)→ Skill（通过 employee_skills 中间表）
    ↓                    ↓
  权限级别              技能类别
  (authority)           (category)
    ↓                    ↓
  工具过滤              模型路由
```

核心关系链：

```
员工 → 绑定技能(employee_skills) → 技能定义(skills) → 工具定义(tool-registry)
                                                              ↓
                                                        agent 可用工具集
```

绑定属性

`employee_skills` 中间表：

字段	说明
`employeeId`	关联员工
`skillId`	关联技能
`proficiency`	熟练度 0-100
`bindingType`	core（核心）/ extended（扩展）/ knowledge（知识）
`boundAt`	绑定时间
`boundBy`	操作人

技能如何影响 Agent 行为

环节	影响方式
工具可用性	只有已绑定技能对应的工具注入 agent
模型选择	主要技能类别决定 LLM 选择（生成类→Claude Sonnet 高温，分析类→Claude Sonnet 低温）
提示词指导	熟练度等级影响系统提示词中的工作风格描述
兼容性约束	技能有 `compatibleRoles` 数组，防止不合适的技能绑定

每个预设员工的技能画像

员工	角色	核心技能 (4个)	主要类别	使用模型
小雷	热点猎手	web_search, trend_monitor, social_listening, heat_scoring	perception	gpt-4o-mini (temp=0.3)
小策	选题策划	topic_extraction, angle_design, audience_analysis, task_planning	analysis	claude-sonnet (temp=0.4)
小资	素材管家	media_search, knowledge_retrieval, news_aggregation, case_reference	knowledge	gpt-4o-mini (temp=0.2)
小文	内容创作	content_generate, headline_generate, style_rewrite, script_generate	generation	claude-sonnet (temp=0.7)
小剪	视频制片	video_edit_plan, thumbnail_generate, layout_design, audio_plan	production	gpt-4o (temp=0.3)
小审	质量审核	quality_review, compliance_check, fact_check, sentiment_analysis	management	claude-sonnet (temp=0.3)
小发	渠道运营	publish_strategy, style_rewrite, translation, audience_analysis	management	claude-sonnet (temp=0.3)
小数	数据分析	data_report, competitor_analysis, audience_analysis, heat_scoring	analysis	claude-sonnet (temp=0.4)

共享技能

部分技能被多个员工共享：

- `audience_analysis`：小策、小发、小数 共用
- `heat_scoring`：小雷、小数 共用
- `style_rewrite`：小文、小发 共用

共享不冲突，每个员工独立维护自己的熟练度。

九、安全与权限保障

多层安全体系

```
Layer 1: 网络层 → Supabase Auth + Middleware
Layer 2: 应用层 → Server Actions requireAuth()
Layer 3: 数据层 → 组织级多租户隔离
Layer 4: Agent 层 → 权限级别工具过滤
Layer 5: 内容层 → 敏感话题审批 + 合规检查
Layer 6: 执行层 → Token 预算 + 审批门控
```

Layer 1: 认证层

Supabase Auth（`src/lib/supabase/`）：

- `client.ts`: 浏览器端 Supabase 客户端
- `server.ts`: RSC/Server Actions 端客户端，使用 cookie 管理会话
- `middleware.ts`: Session 刷新和 cookie 管理

Middleware（`src/middleware.ts`）：

- 每个请求刷新 session cookie
- Dashboard 路由（`/team-hub`, `/analytics`, 等）自动检查认证
- 未认证用户重定向到 `/login`
- 已登录用户访问 `/login` 或 `/register` 重定向到 `/team-hub`

Layer 2: 应用层

Server Actions 认证检查：

所有 Server Actions（`src/app/actions/*`）使用 `requireAuth()` 辅助函数：

```
async function requireAuth() {
  const supabase = await createClient();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) throw new Error("Unauthorized");
  return user;
}
```

任何未认证的调用直接抛出错误，不会执行后续逻辑。

Layer 3: 数据层 — 多租户隔离

组织隔离原则：所有核心表都有 `organizationId` 外键

DAL 层的 `getCurrentUserOrg()` 函数：

```
export async function getCurrentUserOrg(): Promise<string | null> {
  const supabase = await createClient();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) return null;

  const profile = await db.query.userProfiles.findFirst({
    where: eq(userProfiles.id, user.id),
  });
  return profile?.organizationId || null;
}
```

每个 DAL 查询函数都以此为基础过滤数据，确保：

- 组织 A 的用户看不到组织 B 的员工、工作流、消息
- 即使直接调用 DAL 函数，也只返回当前组织的数据

Layer 4: Agent 权限级别

四级权限体系（`authorityLevelEnum`）：

级别	工具权限	适用场景
`observer`	**无工具**	只能观察，不执行任何操作
`advisor`	**仅只读工具**	可搜索、检索，不可生成或修改
`executor`	**所有已绑定工具**	标准执行权限
`coordinator`	**所有已绑定工具**	最高权限，可协调其他员工

只读工具白名单（`assembly.ts`）：

```
web_search, trend_monitor, social_listening, news_aggregation,
knowledge_retrieval, media_search, case_reference, data_report
```

权限过滤发生在 agent 组装阶段，而非执行阶段，从根本上防止越权：

```
// assembly.ts:100-116
if (authority === 'observer') {
  tools = []; // 完全无工具
} else if (authority === 'advisor') {
  tools = tools.filter(t => READ_ONLY_TOOLS.includes(t.name));
}
// executor/coordinator: 保留全部已绑定工具
```

Layer 5: 内容安全

敏感话题机制：

团队配置中定义敏感话题列表（如 `["政治", "军事", "灾难"]`），注入到系统提示词：

```
--- 敏感话题管控 ---
以下话题需要特别审慎，涉及时必须标注并请求人工审批：
- 政治
- 军事
- 灾难

处理规则：
1. 涉及敏感话题时，在输出中明确标注
2. 提供事实依据，避免主观评论
3. 如有疑问，立即暂停并请求人工审核
```

合规检查技能（`compliance_check`）：

- 小审（xiaoshen）的核心技能之一
- 版本 4.0，检测政治、法律、伦理敏感内容
- 在质量审核步骤中自动执行

Layer 6: 执行安全

Token 预算控制：

```
workflowInstances.tokenBudget  // 默认 50000
workflowInstances.tokenUsed    // 累计实际用量

// 每步执行后检查
if (tokenUsed > tokenBudget) {
  throw new Error("Token budget exceeded");
}
```

审批门控：

- 团队级：`approvalRequired` 全局开关
- 步骤级：`approvalSteps` 指定需审批的步骤
- 质量级：`escalationPolicy.qualityThreshold` 低质量自动升级
- 权限级：observer/advisor 输出自动标记 `needs_approval`
- 超时策略：`timeoutAction` 定义超时后的处理方式

核心技能保护：

```
// actions/employees.ts
// 核心技能不可解绑
if (binding.bindingType === 'core') {
  throw new Error("核心技能不可解绑");
}
```

安全矩阵总览

威胁	防御措施	代码位置
未认证访问	Supabase Auth + Middleware 重定向	middleware.ts
越权操作	Server Actions requireAuth()	actions/*.ts
跨租户数据泄露	organizationId 强制过滤	dal/*.ts
Agent 工具越权	权限级别工具过滤	assembly.ts
敏感内容输出	敏感话题提示词 + 合规检查技能	prompt-templates.ts
成本失控	Token 预算强制限制	execute-workflow.ts
低质量输出	质量自评 + 升级策略 + 人工审批	step-io.ts
配置误操作	版本快照 + 回滚支持	employee-versions.ts
核心能力破坏	核心技能不可解绑	actions/employees.ts

总结：系统完成度评估

问题领域	完成度	状态
1. 意图拆解	80%	基础链路完整，缺并行/条件分支
2. 技能学习	60%	绑定和熟练度已实现，自动学习未实现
3. 创建时匹配	95%	预设和自定义两条路径完整
4. 内置技能	100%	28 个技能，6 大类别，定义完备
5. 数据交换	85%	制品传递 + 消息系统完整，缺并行协同
6. 结果判断	85%	自评 + 人工审批 + 驳回重做完整
7. 记忆系统	70%	存储和注入完整，缺自动写入和淘汰
8. 员工-技能关系	95%	关系模型清晰，兼容性检查到位
9. 安全权限	90%	六层防御体系完整，缺 RLS 细粒度控制